

MEDICAL MONITORING AND ADVANCED SENSOR DATA PROCESSING – TASK 05: AUTOENCODERS AND GANS FOR DENOISING

1 General Information

The practical experiments should be done in groups of up to three people. Each group must submit one solution via Digicampus. Of course, you can discuss the tasks between groups, but forwarding/copying other students' source code is forbidden.

Prepare the task until the deadline provided below/in Digicampus.

On Digicampus, there is a template for you to prepare your code for submission. Please complete your code within this template. The **README.md** file provides information on how to set up your Python environment for running your code. A Jupyter notebook example is also provided. You can adapt the content of this Jupyter notebook for preparing your solution.

Before uploading your solution, please make sure that the following requirements are met:

- Prepare your result documentation as Jupyter Notebook and a report
- Print a PDF version of your Jupyter notebook AND add it to the .zip archive you upload on Digicampus.
- All source code must be available and executable for us (this implies that you specify the required packages/libraries. In addition, you have to provide necessary data (ideally data should be in a subfolder, and you use relative paths) and/or give advice if paths have to be adjusted; in order to do so, you should provide a reasonable README file)
- **GREEN** sections in the chapter are tasks. Please include a numbering according to the questions' number Q<xy>
- List your sources (if applicable)

Submission deadline is **Monday, 11.08.2025, 08:00**; submission is done via **Digicampus**.

Please ask in case of any doubts regarding the contents or the formal procedure.

2 Elements to be delivered

1. Jupyter Notebook: the students must deliver the code (as .ipynb file) they developed to visualize, analyse and plot the data according to the template provided.
A pdf version of the Jupyter notebook with the results of the notebook's run should be provided as a backup in case the notebook fails to be run again.
2. The students' recordings that are necessary to execute their Jupyter notebooks.
3. Report: the students must deliver a report (as .pdf file) using the ReportTemplate.docx found in Digicampus. The report must contain the following:
Title Page
Title: Clear and specific
Course name / lab name
Students' names and emails
Date
Answer to the questions (with the appropriate question numbering)
4. When a question asks to discuss a topic, the word limit is an upper limit.

Notes:

- Please submit only the required and necessary files. Do not include any files that are NOT explicitly requested or that are not essential for executing your code.
- For this exercise, you have different elements to be delivered in each question (Q), in addition to the corresponding report.

3 Remarks on the Hardware and software

A documentation to perform the recordings based on the smartwatch and Biopac platform is provided [1]. Executables to perform recordings via the ADI Study watch are provided in a Flashdrive [2]. Those executables should be copied to the computer which will record the data.

3.1 Biopac

One can export the acquired data in a format that is suitable for postprocessing.

3.2 ADI Study smartwatch

The communication between the smartwatch and the computer is established via a Dongle connected to a plugged USB port. One needs to check the port address (COM) using the device manager.

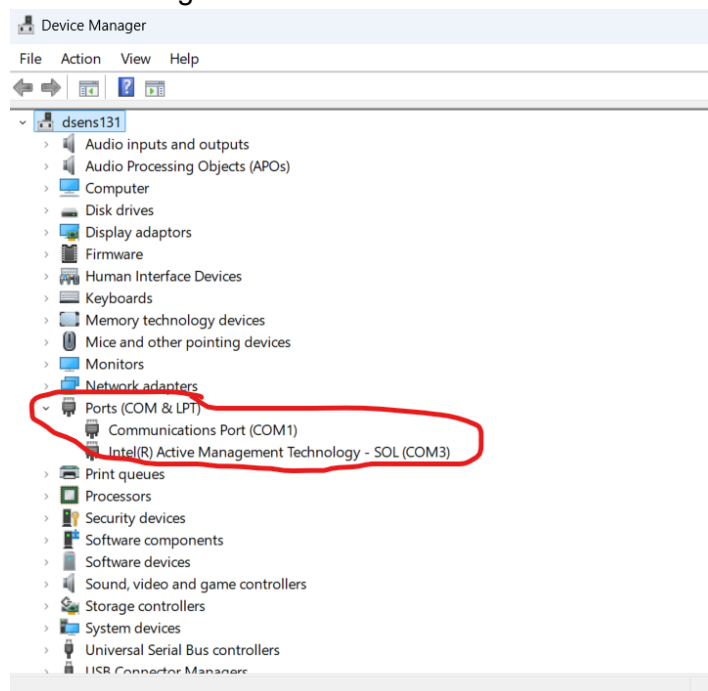


Figure 1 Device manager (on Windows).

The recordings are stored on the computer in .csv files.

The smartwatch can provide an electrocardiographic signal (ECG), a photoplethysmographic signal (PPG), and accelerometer signals (along the 3 directions).

4 Remarks on reports and data collection

Following some remarks on the report and data collection:

- Reports for data gathering should be detailed and complete. Students will use the data they collect to work on it by themselves.
- The report must assure the reproducibility of the experiment and data gathering, bearing in mind the amount and quality of data that is intended to be collected.
- If you cite from the literature, use the APA citation style
- For plots in the time domain, signals must be plotted with x-axis in time, in seconds.
- All plots must have axes labelled in a meaningful way.
- All plots or images must have caption written in a meaningful way.
- In the caption, do not forget to refer to the question number **Q...** the plot is built for.
- If a question asks you to make a plot, include the plots with appropriate reference to the question in the caption, and write a reference to the plot in the report.

For example:

Qxx. Area under the curve obtained on the test set by several classifiers is displayed on Figure 2.

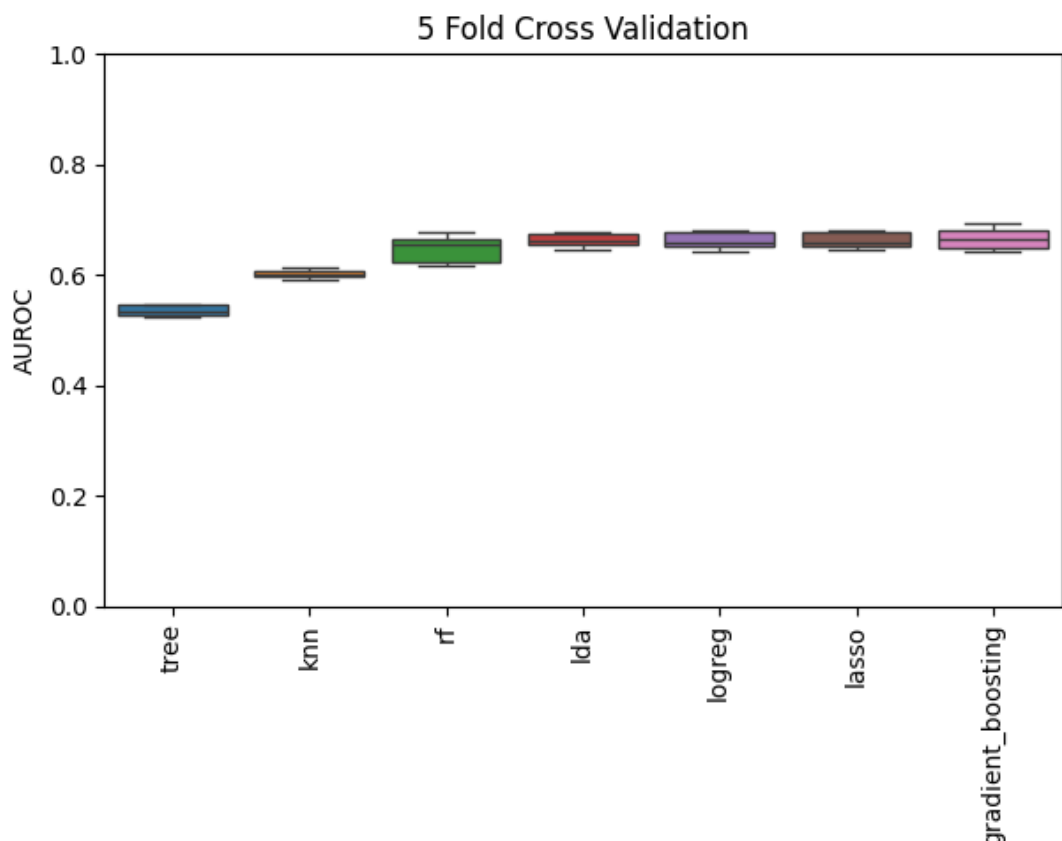


Figure 2 Area under the curve of several classifiers on the test set (**Qxx**).

- **ATTENTION:** Have a look at the general document on our expectation and common mistakes, which is available in Digicampus

5 Tasks

5.1 Notes

For the following tasks, you will require ECG signals recorded during previous exercises. Please ensure that you use signals obtained from the same individual (subject). Take into account all the comments and observations provided in your reports for previous exercises. You are encouraged to ask questions if any aspect is unclear. In this exercise, you will use clean ECG signals for model training, while data recorded under the "Stairs", "Workout", and added power electrical noise conditions will be used for testing. Utilize ECG data collected from both the Biopac system and the smartwatch. PPG and acceleration data are not required for this exercise.

5.2 Questions

Q1 (2 points): Dataset construction

- a) Data collection and resampling. Consider two sets of ECG data recorded using the Biopac system and a smartwatch: (1) "clean" data (recorded in the last session of data acquisition) and (2) noisy data, which includes recordings from the "Stairs" and "Workout" conditions, as well as data with added power line electrical noise (recorded in the second and last session). This noisy dataset will henceforth be referred to as "Experimental Noisy Data" (END) and will serve as a secondary testing subset. Resample all signals to match the lower sampling rate among the two devices. Collect as much ECG data as possible (from the already recorded data), as a larger dataset will enhance the training of your model.
- b) Filtering. Filter the "clean" signals to maximize signal quality while retaining relevant physiological information. Specifically, remove 50 Hz power line noise, baseline wander, high-frequency noise, and any other potential sources of interference. Apply filtering techniques covered in previous lectures. Plot four randomly selected examples, showing both the original and the corresponding filtered signals. Include in your report a detailed description of your implemented filters with their characteristics.
- c) Segmentation. Segment both the filtered clean data and the END data using a 15-second window with 50% overlap. The END data will not be used further until the final stages of the analysis.
- d) Dataset splitting. Divide the set of clean segments into two subsets: 90% for training and 10% for testing. For example, if you have 10,000 segments, allocate 9,000 to the training set and 1,000 to the test set. Clearly report the number of segments in each subset.
- e) Noise injection. For both the training and testing subsets of clean signals, artificially add the following types of noise to each segment: baseline wander, power line noise (50 Hz), electromyographic noise, electrode contact loss, missing data (flat line), and any other relevant noise types observed in your noisy dataset. Each clean segment must be used to generate eight distinct noisy versions. Consider implementing a function that randomly combines different types of noise, potentially varying the number and type of noise sources applied to each segment.

Note: Noise must be generated artificially. The more realistically you simulate actual noise, the more effective subsequent analysis will be. The data contaminated with

noise in this step will be referred to as “noisy data”. You should research about the best practices to simulate the different types of noise you want to include.

- f) Saving data into csv files. Save the resulting data from the previous step into two csv files: “clean_and_noisy_train.csv” and “clean_and_noisy_test.csv”. The data should be structured in rows for instances and columns with data and labels with the following four headers:

signal_id → A composite index indicating the segment number and the noisy version

original_id → The main index corresponding to the original segment number

signal_type → Indicates whether the signal is “clean” or “noisy”

data → The signal data for the corresponding segment

Refer to Figure 1 at the end of this section for an example of the required format.

Note: For the END dataset, only the “data” header is relevant. The other headers are not necessary for subsequent processing; however, as with the training and testing datasets, each row should represent a data instance (i.e., a signal segment). Save this data into the file “END_test.csv”.

- g) Plotting examples. Plot two random examples from the training set and two random examples from the test set. Each plot should display the original signal alongside its eight noisy versions.

Discussion

Write a comprehensive discussion of at least one full page (maximum one and a half pages) regarding the implementation of Q1. While you may expand on any relevant aspects, your discussion must address at least the following points:

- The types and characteristics of noise identified in both your clean and noisy data
- The various filters implemented for signal processing. Explain the types of filters used, their configurations, and the rationale for your choices and parameter settings. (Q1b)
- The different types of noise injected into the clean signals. Describe your simulation methods and justify your selection of noise types and their implementation. (Q1e)
- An explanation of the plots provided, including how they relate to your data processing and noise injection procedures

Deliverables of task Q1:

- Jupyter notebook: “exercise_5_Q1.ipynb” and “exercise_5_Q1.pdf”
- csv files: “clean_and_noisy_train.csv”, “clean_and_noisy_test.csv”, and “END_test.csv”.

	signal_id	original_id	signal_type	data
1	clean_0		0 clean	-0.0647,-0.0582,-0.0513,-0.0442,-0.0365,-0.0283,-0.0194...
2	noisy_0_0		0 noisy	-0.1018,-0.0506,-0.0132,-0.047,-0.075,-0.0303,0.0188,-0....
3	noisy_0_1		0 noisy	-0.0664,-0.0341,-0.0492,-0.0127,-0.0106,-0.0073,0.0047,...
4	noisy_0_2		0 noisy	-0.0588,-0.0216,-0.0611,-0.1143,-0.0162,-0.0892,0.0813,...
5	noisy_0_3		0 noisy	-0.0868,-0.0212,-0.0715,-0.0299,-0.0487,-0.0304,-0.0212...
6	noisy_0_4		0 noisy	-0.0724,-0.0343,0.0158,-0.0737,-0.0358,0.019,0.1263,0.0...
7	noisy_0_5		0 noisy	-0.0777,-0.0576,-0.0492,-0.0324,-0.0334,-0.0476,-0.0354...
8	noisy_0_6		0 noisy	0.02,-0.0232,-0.0978,-0.0777,-0.0545,-0.0402,0.0172,-0....
9	clean_1		1 clean	0.0713,0.0821,0.094,0.1072,0.1217,0.1374,0.1544,0.172...
10	noisy_1_0		1 noisy	0.1048,-0.0296,0.0121,0.1035,0.195,0.1593,0.0216,0.097...
11	noisy_1_1		1 noisy	0.2082,-0.0037,0.1933,0.0456,0.0783,0.2372,0.1062,0.24...
12	noisy_1_2		1 noisy	0.129,0.0957,-0.0468,0.0949,0.1628,0.1934,0.1463,0.061...
13	noisy_1_3		1 noisy	0.0652,0.0535,0.0433,0.0651,0.1072,0.1312,0.1257,0.12...
14	noisy_1_4		1 noisy	0.1016,0.0551,0.0604,0.1299,0.1582,0.1193,0.1157,0.18...
15	noisy_1_5		1 noisy	0.0525,-0.0303,-0.0855,-0.0202,0.0935,0.1184,0.0418,-0...
16	noisy_1_6		1 noisy	0.0704,0.0841,0.0961,0.09,0.1603,0.1872,0.151,0.1536,0...
17	clean_2		2 clean	-0.0158,-0.0157,-0.0154,-0.015,-0.0146,-0.0141,-0.0135,-...
18	noisy_2_0		2 noisy	-0.0876,0.0733,0.0957,0.0558,0.0002,-0.2209,-0.19,-0.06...
19	noisy_2_1		2 noisy	-0.0808,-0.0571,-0.0393,-0.0516,-0.0767,-0.0793,-0.0552...

Fig 1. Example of the required csv file format

Q2 (2 points): generative adversarial network (GAN) for denoising

This exercise challenges you to implement and evaluate a Generative Adversarial Network (GAN) for denoising Electrocardiogram (ECG) signals. You will gain hands-on experience with deep learning architectures, data preprocessing, model training, and performance evaluation in the context of biomedical signal processing and denoising, in particular. You are expected to implement a GAN using either TensorFlow or PyTorch libraries. Ensure your code is well-commented, modular, and adheres to good programming practices.

- a) Loading data. Load the data for training and testing, from the csv files saved in the question Q1: "clean_and_noisy_train.csv" and "clean_and_noisy_test.csv". Implement a function to read these csv files and reconstruct the structured data, where each original_id maps to its clean signal and a list of noisy_versions.

Hint: The data column in the csv contains comma-separated string representations of the signal arrays. You will need to parse these back into numpy arrays.

- b) Data preprocessing: prepare your raw ECG signals for GAN training. Neural networks often perform better with normalized inputs. Apply MinMax Scaling to both your clean and noisy training signals. Scale the training set to the range $[-1, 1]$; fitting the MinMaxScaler on the entire training set. This range is often preferred for GANs, especially with tanh activation in the generator's output layer, as it helps in stabilizing training.

Crucial Step: Fit the MinMaxScaler on the entire training dataset, which means combining all clean signals and all noisy signals from your training set. This step is essential to capture the full range of amplitude variations (including those introduced by noise) present in your training data. Once fitted, use this same learned scaler to

transform all clean and noisy signals in both your training and testing sets to the range $[-1, 1]$.

Consideration: Each original clean signal has multiple noisy versions. Ensure your scaling process handles this structure correctly.

- c) Data preparation: organize your loaded and scaled data into a format suitable for creating paired datasets. Create a Python dictionary or similar data structure that maps each `original_id` from your training set to its scaled clean signal and a list of its corresponding scaled noisy versions. This mapping is crucial for correctly pairing noisy inputs with their ground-truth clean outputs.
- d) Data preparation: construct the actual input-output pairs for your GAN. From the mapping created in part (c), generate two lists or arrays: one containing all the scaled noisy signals, and another containing their corresponding scaled clean signals. Each noisy signal should be paired with its "ground truth" clean version. Convert these lists into `tf.data.Dataset` objects (if using TensorFlow) or `torch.utils.data.Dataset` and `DataLoader` (if using PyTorch).

Configure your dataset for training:

Batching: group samples into batches (e.g., `BATCH_SIZE = 32`)

Shuffling: shuffle the dataset to ensure proper generalization and prevent the model from learning the order of samples

- e) GAN architecture – Generator (G): design a neural network that takes a noisy ECG segment as input and generates a denoised ECG segment. Build the Generator model with `Conv1D` for downsampling (encoder) and `Conv1DTranspose` for upsampling (decoder) is recommended.

Skip connections: incorporate skip connections between the encoder and decoder blocks (it is not mandatory, but you might play with skip connections).

Input shape: the input shape should be `(segment_length, 1)`, representing time steps and a single channel for ECG.

Activation functions: use `LeakyReLU` in intermediate layers for the encoder and `ReLU` for the decoder. The final output layer must use a `tanh` activation function to ensure the generated signals are within the $[-1, 1]$ scaled range.

Batch normalization: include `BatchNormalization` layers after convolutional layers (except for the input and output layers) to stabilize training.

Output shape: ensure the generator's output shape perfectly matches the `(segment_length, 1)` of the clean ECG signals. Implement padding or cropping if necessary to achieve the exact desired output length from `Conv1DTranspose` operations.

- f) GAN architecture – Discriminator (D): Design a neural network that can distinguish between real (clean) ECG signals and fake (generated by G) ECG signals. Build the Discriminator model using `Conv1D` layers for downsampling. Input Shape: The input shape should be `(segment_length, 1)`, identical to the clean ECG signals and the generator's output. Activation Functions: use `LeakyReLU` in intermediate layers. The final output layer should use a `sigmoid` activation function to produce a probability score (0 for fake, 1 for real).

Batch Normalization: include `BatchNormalization` layers.

Regularization (optional but recommended): consider adding Dropout layers to the discriminator to prevent overfitting and improve training stability. Experiment with dropout rates (e.g., 0.2-0.3).

Instantiate Models: Instantiate both your Generator and Discriminator models after defining their architectures.

g) Define optimizers and loss functions.

Optimizers: use the Adam optimizer for both the Generator and Discriminator. Learning rates: Crucially, set different learning rates for G and D. A common practice for stable GAN training is to have the Discriminator learn slower than the Generator (e.g., Discriminator LR = $1e-4$, Generator LR = $2e-4$). Justify why this imbalance in learning rates can be beneficial. Beta 1: set $\beta_1=0.5$ for both Adam optimizers. Loss functions:

Adversarial loss (for Discriminator and Generator): use BinaryCrossentropy for the adversarial component of the loss.

Label smoothing (optional but recommended for discriminator): implement label smoothing for the discriminator's real labels (e.g., target 0.9 instead of 1.0) and fake labels (e.g., target 0.1 instead of 0.0).

L1 loss (for Generator): use Mean Absolute Error (MAE) as an additional loss component for the Generator. This is often referred to as the "pixel-wise" or "content" loss.

Generator's total loss: define the Generator's loss as a weighted sum of the adversarial loss and the L1 loss. A common weighting factor (λ_{l1}) for the L1 loss is 100.

h) Training: implement the training logic for your GAN. Define a `train_step` function to compile it. Inside `train_step`, use `tf.GradientTape` (TensorFlow) or manually compute gradients (PyTorch) to:

Discriminator training: generate fake ECG signals using the Generator (with `training=False` for the Generator during this phase).

Pass both real clean signals and generated fake signals to the Discriminator.

Calculate the Discriminator's loss based on its ability to correctly classify real as real and fake as fake.

Compute gradients and apply them to the Discriminator's trainable variables.

Crucial: Temporarily set `discriminator.trainable = True` only during its update, and `False` otherwise, especially when updating the Generator.

Generator training: generate fake ECG signals using the Generator (with `training=True` for the Generator).

Pass the generated signals to the Discriminator (with `training=False` for the Discriminator – its weights are fixed during G's update).

Calculate the Generator's loss, aiming to fool the Discriminator and minimize the L1 distance to the true clean signals.

Compute gradients and apply them to the Generator's trainable variables.

Crucial: ensure `discriminator.trainable = False` when training the Generator.

Implement the overall training loop across multiple epochs: iterate through your `tf.data.Dataset` (or PyTorch `DataLoader`) in batches.

Training schedule: implement a strategy where the Discriminator is trained once, and then the Generator is trained multiple times (e.g., `G_UPDATE_FREQ = 2` or `3`). Keep track of the average Discriminator and Generator loss for each epoch.

Print the loss values per epoch in the format: Epoch X/Y | D Loss: d.dddd | G Loss: g.gggg.

Pre-building optimizers: perform dummy `apply_gradients` calls to your optimizers to ensure their internal variables are built. This prevents `ValueError` inside the compiled function.

- i) Plot training history and save losses. Store the discriminator and generator loss values for each epoch during training. After training, plot both loss histories on a single graph. Label your axes ("Epoch", "Loss") and provide a clear title ("GAN Training Loss History"). Save this plot as an image file (e.g., `gan_loss_history.png`). Save the loss history data (epoch, D loss, G loss) into a csv file.
- j) Architecture visualization. Generate a visual representation of both your Generator and Discriminator architectures. Use `tf.keras.utils.plot_model` (TensorFlow) or a similar utility (PyTorch, e.g., `torchviz`) to save the model diagrams as PNG files (e.g., `generator_architecture.png`, `discriminator_architecture.png`). Ensure `show_shapes=True` to display layer input/output shapes.
- k) Testing and evaluation: quantitatively assess the performance of your trained GAN on unseen data. Load the "clean_and_noisy_test.csv" dataset. Use your trained Generator to denoise the noisy signals from the test set. Remember to apply the same `MinMaxScaler` you used for training to the test signals before feeding them to the Generator, and then inverse transform the Generator's output back to the original amplitude scale. Calculate the Mean Absolute Error (MAE) and the Peak Signal-to-Noise Ratio (PSNR) between the generated denoised signals and the true clean signals in the test set and compute the overall average MAE and PSNR.
- l) Visualizing denoising on samples from the test set. Generate a table with MAE and PSNR values for ten randomly selected signals from the test set. For each selected signal, create a figure with three subplots:
 - Original Clean ECG (Ground Truth): plot the true clean signal
 - Noisy ECG Input: plot the corresponding noisy signal that was fed to the Generator
 - Generated (Denoised) ECG: plot the output of your Generator after inverse scaling
- m) Architecture and parameter adjustment. Based on your evaluation in (k) and (l), discuss potential areas for improvement. Then, experiment with one or more of the following adjustments to improve your GAN's performance:
 - Change filter sizes, kernel sizes, or strides in G or D
 - Modify the `lambda_l1` weighting factor in the generator loss
 - Adjust learning rates further, or experiment with learning rate schedulers
 - Explore different regularization techniques (e.g., more Dropout, L2 regularization)
 - Increase the number of training epochs significantly

Describe the change you made, the rationale behind it, and the observed impact on MAE/PSNR and visual quality. Provide updated metrics and a sample plot to demonstrate the improvement.

- n) Saving models and scaler. Save your trained Generator and Discriminator models in the native Keras format (.keras) or PyTorch's recommended format (.pt or .pth). Create a dedicated directory (e.g., trained_gan_models) for this.
 Crucially, save the MinMaxScaler object you used for preprocessing. joblib is an excellent library for this (joblib.dump(scaler, 'minmax_scaler.joblib')).
- o) Loading models, scaler, and END data: demonstrate the reusability of your trained models and preprocessing tools. In a new Jupyter Notebook, load the previously saved Generator model and the MinMaxScaler. Load the new external dataset, END_test.csv. Verify that all components (model, scaler, data) are loaded successfully.
- p) Testing on END data. Apply the loaded Generator and Scaler to filter (denoise) the signals from the END_test.csv dataset. Randomly select ten signals from the END_test set. For each selected signal, create a figure with two subplots:
- Noisy ECG Input: plot the original noisy ECG signal from the END_test set.
 - Generated (Denoised) ECG: plot the denoised output from your Generator (after inverse scaling).

Since you do not have ground-truth clean signals for the END_test data, you cannot calculate MAE or PSNR. Focus on the visual quality of the denoised output and discuss its characteristics.

Discussion

Write a comprehensive discussion of at least one full page (maximum one and a half pages) regarding the implementation of Q2. Consider all the elements that were mentioned in the instructions to be discussed.

While you may expand on any relevant aspects, your discussion must also address at least the following points:

- Explain why using the same scaler fitted on clean training data is important for both training and testing sets (Q2b)
- Explain why creating paired datasets is essential for this denoising GAN architecture, as opposed to unpaired GANs (Q2d)
- The importance of data structure for training the GAN (Q2e)
- The GAN architecture and its influence on the performance. Discuss Discriminator and Generator structures and how they might affect the final outcome. (Q2e)
- Explain the purpose and benefit of skip connections in this context (e.g., preserving fine-grained details) (Q2e)
- The role of optimizers and loss functions
- Explain the role of beta_1 in Adam and why 0.5 is often chosen for GANs (Q2g)
- Explain how label smoothing helps stabilize GAN training (Q2g)
- Explain why combining adversarial loss with an L1 loss is particularly effective for image/signal translation tasks like denoising (Q2g)
- The training process. Generate a mind map to describe in a general manner the entire process of training a GAN for denoising ECG signals.
- Explain why training the Generator more frequently than the Discriminator can be beneficial (Q2h)
- How the different parameters influence GAN performance

- Describe the change you made, the rationale behind it, and the observed impact on MAE/PSNR and visual quality. Provide updated metrics and a sample plot to demonstrate the improvement (mentioned also in Q2m)
- Explain why saving the scaler is vital for future inference (Q2n)
- What further actions would you take to improve your implementation?

Deliverables of task Q2:

- Jupyter notebooks:
“exercise_5_Q2a.ipynb” and “exercise_5_Q2a.pdf” → for items a to n
“exercise_5_Q2b.ipynb” and “exercise_5_Q2b.pdf” → for items o and p
- Files: discriminator model, generator model, MinMaxScaler, discriminator architecture (picture format), generator architecture (picture format), GAN loss history (picture format), loss history data (epoch, D loss, G loss) (csv).

Q3 (2 points)

This exercise guides you through the implementation and evaluation of a deep autoencoder for denoising electrocardiogram (ECG) signals. You will explore a different deep learning paradigm compared to GANs, focusing on direct signal reconstruction. You are expected to implement the autoencoder using either TensorFlow or PyTorch. Ensure your code is well-commented, modular, and adheres to good programming practices.

- a) Data loading: load the ECG datasets for training and testing from the csv files "clean_and_noisy_train.csv" and "clean_and_noisy_test.csv". Reuse or adapt the data loading function from Q2 to reconstruct the structured data (original_id, clean signal, noisy versions).
- b) Data preprocessing: prepare your raw ECG signals for autoencoder training. Apply MinMax scaling to both your clean and noisy training signals, scaling them to the range [-1, 1]; fitting the MinMaxScaler on the entire training set.
 Crucial Step: Fit the MinMaxScaler on the entire training dataset, which means combining all clean signals and all noisy signals from your training set. This step is essential to capture the full range of amplitude variations (including those introduced by noise) present in your training data. Once fitted, use this same learned scaler to transform all clean and noisy signals in both your training and testing sets to the range [-1, 1].
 Consideration: Ensure your scaling process correctly handles the multiple noisy versions per clean signal.
- c) Data preparation: organize your loaded and scaled data into a format suitable for creating paired datasets. Create a Python dictionary or similar data structure that maps each original_id from your training set to its scaled clean signal and a list of its corresponding scaled noisy versions. This mapping is crucial for correctly pairing noisy inputs with their ground-truth clean outputs.
- d) Data preparation: construct the input-output pairs for your autoencoder. From the mapping created in part (c), generate two lists or arrays: one containing all the scaled noisy signals (these will be the inputs to the autoencoder), and another containing their corresponding scaled clean signals (these will be the targets/outputs the autoencoder tries to reconstruct). Convert these lists into tf.data.Dataset objects (if using TensorFlow) or torch.utils.data.Dataset and DataLoader (if using PyTorch).
 Configure your dataset for training:
 - Batching: group samples into batches (e.g., batch_size = 32)
 - Shuffling: shuffle the dataset to ensure proper generalization
 - Self-reflection: explain why an autoencoder for denoising is inherently a supervised learning task, unlike the adversarial training of a GAN
- e) Autoencoder architecture: design an autoencoder that takes a noisy ECG segment as input and reconstructs its clean counterpart. Build the autoencoder model. This will typically consist of an encoder (downsampling path using Conv1D layers) and a decoder (upsampling path using Conv1DTranspose layers).
 - Skip connections: incorporate skip connections between corresponding encoder and decoder blocks (similar to a u-net architecture - it is not mandatory, but you might play with skip connections).
 - Input shape: the input shape should be (segment_length, 1).

- Activation functions: use LeakyReLU in intermediate layers for the encoder and ReLU for the decoder. The final output layer must use a tanh activation function to ensure the reconstructed signals are within the $[-1, 1]$ scaled range, matching your preprocessing.
 - Batch normalization: include BatchNormalization layers after convolutional layers (except for the input and output layers) to stabilize training.
 - Output shape: ensure the autoencoder's output shape perfectly matches the (segment_length, 1) of the clean ECG signals. Implement padding or cropping if necessary to achieve the exact desired output length from Conv1DTranspose operations.
- f) Define optimizer and loss function
- Optimizer: use the Adam optimizer for the autoencoder. A standard learning rate (e.g., $1e-4$ or $1e-3$) is usually a good starting point.
 - Loss function: for a denoising autoencoder, the primary goal is to reconstruct the clean signal from the noisy input. Therefore, the loss function should measure the similarity between the autoencoder's output and the true clean signal.
 - Use mean absolute error (MAE) or mean squared error (MSE) as the reconstruction loss.
- g) Training: implement the training logic for your autoencoder. Define a train_step function to compile it. Inside train_step, use tf.gradienttape (TensorFlow) or manually compute gradients (PyTorch) to:
- Pass noisy ECG signals as input to the autoencoder.
 - Get the reconstructed (denoised) output.
 - Calculate the reconstruction loss between the reconstructed output and the true clean signal.
 - Compute gradients and apply them to the autoencoder's trainable variables.
 - Implement the overall training loop across multiple epochs:
 - Iterate through your tf.data.dataset (or pytorch dataloader) in batches.
 - Keep track of the average reconstruction loss for each epoch.
 - Print the loss values per epoch in the format: epoch x/y | reconstruction loss: l.llll.
 - Pre-building Optimizer: perform dummy apply_gradients calls to your optimizer to ensure its internal variables are built.
- h) Plot training history and save losses. Store the reconstruction loss values for each epoch during training. After training, plot the loss history on a graph. Label your axes ("epoch", "loss") and provide a clear title ("autoencoder training loss history"). Save this plot as an image file (e.g., autoencoder_loss_history.png). Save the loss history data (epoch, reconstruction loss) into a csv file.
- i) Architecture visualization. Generate a visual representation of your autoencoder architecture. Use tf.keras.utils.plot_model (TensorFlow) or a similar utility (PyTorch) to save the model diagram as a PNG file (e.g., autoencoder_architecture.png). Ensure show_shapes=true.
- j) Testing and evaluation: quantitatively assess the performance of your trained autoencoder on unseen data. Load the ecg_dataset_clean_and_noisy_test.csv dataset. Use your trained autoencoder to denoise the noisy signals from the test set.

Remember to apply the same MinMaxScaler you used for training to the test noisy signals before feeding them to the autoencoder, and then inverse transform the autoencoder's output back to the original amplitude scale. Calculate the Mean Absolute Error (MAE) between the generated denoised signals and the true clean signals in the test set. Calculate the Mean Absolute Error (MAE) and the Peak Signal-to-Noise Ratio (PSNR) between the generated denoised signals and the true clean signals in the test set and compute the overall average MAE and PSNR.

k) Visualizing denoising on samples from the test set. Generate a table with MAE and PSNR values for ten randomly selected signals from the test set. For each selected signal, create a figure with three subplots:

- Original Clean ECG (Ground Truth): plot the true clean signal
- Noisy ECG Input: plot the corresponding noisy signal that was fed to the autoencoder
- Generated (autoencoder output) ECG: plot the output of your autoencoder after inverse scaling

l) Architecture and parameter adjustment. Based on your evaluation in (j) and (k), discuss potential areas for improvement. Then, experiment with one or more of the following adjustments to improve your autoencoder's performance:

- Change filter sizes, kernel sizes, or strides in the encoder/decoder.
- Adjust the learning rate or experiment with learning rate schedulers.
- Explore different regularization techniques (e.g., dropout, L2 regularization).
- Increase the number of training epochs significantly.
- Experiment with different loss functions (e.g., Huber loss).
- Describe the change you made, the rationale behind it, and the observed impact on MAE/PSNR and visual quality. Provide updated metrics and a sample plot to demonstrate the improvement.

m) Saving model and scaler. Save your trained autoencoder model in the native Keras format (.keras) or PyTorch's recommended format (.pt or .pth). Create a dedicated directory (e.g., trained_autoencoder_models) for this. Crucially, save the MinMaxScaler object you used for preprocessing. joblib is an excellent library for this.

n) Loading model, scaler, and END data: demonstrate the reusability of your trained autoencoder and preprocessing tools. In a new Jupyter Notebook, load the previously saved autoencoder model and the MinMaxScaler. Load the new external dataset, END_test.csv. Verify that all components (model, scaler, data) are loaded successfully.

o) Testing on END data. Apply the loaded autoencoder and scaler to filter (denoise) the signals from the END_test.csv dataset. Randomly select ten signals from the END_test set. For each selected signal, create a figure with two subplots:

noisy ECG input: plot the original noisy ECG signal from the END_test set

denoised ECG (autoencoder output): plot the denoised output from your autoencoder (after inverse scaling)

Since you do not have ground-truth clean signals for the END_test data, you cannot calculate MAE or PSNR. Focus on the visual quality of the denoised output and discuss its characteristics.

Discussion

Write a comprehensive discussion of at least one full page (maximum one and a half pages) regarding the implementation of Q3. Consider all the elements that were mentioned in the instructions to be discussed.

While you may expand on any relevant aspects, your discussion must also address at least the following points:

- The importance of data structure for training the autoencoder (Q3b,c,&d)
- The autoencoder architecture and its influence on the performance (Q3e)
- Explain how skip connections are particularly beneficial in denoising autoencoders for preserving high-frequency details (Q3e)
- The role of optimizers and loss functions (Q3f)
- Explain the difference between MAE and MSE and why one might be preferred over the other for signal reconstruction with autoencoders (Q3f)
- The training process. Generate a mind map to describe in a general manner the entire process of training an autoencoder for denoising ECG signals (Q3g)
- Compare the training processes between the previously implemented GAN and the autoencoder here
- How the different parameters influence autoencoder performance
- Explain why saving the scaler is vital for future inference (Q3m)
- What further actions would you take to improve your implementation?

Deliverables of task Q3:

- Jupyter notebooks:
 "exercise_5_Q3a.ipynb" and "exercise_5_Q3a.pdf" → for items a to m
 "exercise_5_Q3b.ipynb" and "exercise_5_Q3b.pdf" → for items n and o
- Files: autoencoder model, MinMaxScaler, autoencoder architecture (picture format), autoencoder loss history (picture format), loss history data (epoch, D loss, G loss) (csv)

6 Material (and recommended readings)

- [1] Souai, Wiem. "The Complete Guide to Generative Adversarial Networks [GANs]." Medium, 8 Feb. 2024, medium.com/ubiai-nlp/the-complete-guide-to-generative-adversarial-networks-gans-22a6afa57f03.
- [2] Candido, Renato. Generative Adversarial Networks: Build Your First Models. 16 Aug. 2024, realpython.com/generative-adversarial-networks. Accessed 11 July 2025.
- [3] GeeksforGeeks. "Generative Adversarial Network (GAN)." GeeksforGeeks, 2 June 2025, www.geeksforgeeks.org/deep-learning/generative-adversarial-network-gan. Accessed 11 July 2025.
- [4] "Intro to Autoencoders." TensorFlow, www.tensorflow.org/tutorials/generative/autoencoder. Accessed 11 July 2025.
- [5] Rosebrock, Adrian. "Denoising Autoencoders With Keras, TensorFlow, and Deep Learning - PyImageSearch." PyImageSearch, 17 Apr. 2021, pyimagesearch.com/2020/02/24/denoising-autoencoders-with-keras-tensorflow-and-deep-learning. Accessed 11 July 2025.

- [6] Chollet, By Francois. Building Autoencoders in Keras. blog.keras.io/building-autoencoders-in-keras.html. Accessed 11 July 2025.